

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

"C++ OOP / PARALLEL"

Реквизиты

Дмитрий Малов, Z33433, 2025 год

Цель работы

Познакомить студента с принципами объектно-ориентированного программирования на примере создания сложной синтаксической структуры. Придумать синтаксис своего персонального мини-языка параллельного программирования, а также реализовать его разбор и вычисление.

Задача 1

[C++ PARALLEL LANG]

Создать параллельный язык программирования. Требуется создать язык программирования, в котором будет доступна установка следующих команд:

- Установка счетного цикла
- Вывод в консоль
- Вывод в файл в режиме добавления
- Арифметические операции $+$, $-$, $*$, $/$

Счетный цикл должен поддерживать дальнейшую установку всех остальных поддерживаемых команд. Для реализации задачи использовать технологию объектно-ориентированного программирования в части реализации поддерживаемых команд языка.

Решение задачи 1

Базовый класс Command

Абстрактный класс, определяющий интерфейс для всех команд:

- Виртуальный метод `execute()` для выполнения команды
- Виртуальный деструктор для корректного удаления объектов

```
// базовый класс
class Command {
public:
    virtual void execute() = 0;
    virtual ~Command() = default;
};
```

Класс PrintCommand

Вывод сообщения в консоль.

- Хранит строку для вывода
- Использует мьютекс для потокобезопасности

```
//ВЫВОД В КОНСОЛЬ
class PrintCommand : public Command {
    std::string message;
public:
    PrintCommand(const std::string& msg) : message(msg) {}
    void execute() override {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << message << std::endl;
    }
};
```

Класс FileAppendCommand

Запись в файл

- Хранит имя файла и содержимое
- Открывает файл в режиме добавления

```
//ВЫВОД В ФАЙЛ
class FileAppendCommand : public Command {
    std::string filename;
    std::string content;
public:
    FileAppendCommand(const std::string& file, const std::string& cnt)
        : filename(file), content(cnt) {}
    void execute() override {
        std::ofstream file(filename, std::ios::app);
        if (file) {
            file << content << std::endl;
        }
    }
};
```

Класс MathCommand

Арифметические операции

- Хранит два операнда и оператор
- Поддерживает операции +, -, *, /

```
//арифметические операции
class MathCommand : public Command {
    double a, b;
    char op;
public:
    MathCommand(double x, double y, char operation)
        : a(x), b(y), op(operation) {}
    void execute() override {
        double result = 0;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/': result = (b != 0) ? a / b : 0; break;
        }
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << "Math: " << a << " " << op << " " << b << " = " << result << std::endl;
    }
};
```

Класс LoopCommand

Циклическое выполнение

- Хранит количество итераций
- Содержит вектор вложенных команд
- Реализует метод addCommand() для добавления команд в цикл

```
//цикл
class LoopCommand : public Command {
    int iterations;
    std::vector<std::unique_ptr<Command>> commands;
public:
    LoopCommand(int iter) : iterations(iter) {}
    void addCommand(std::unique_ptr<Command> cmd) {
        commands.push_back(std::move(cmd));
    }
    void execute() override {
        for (int i = 0; i < iterations; ++i) {
            for (auto& cmd : commands) {
                cmd->execute();
            }
        }
    }
};
```

Парсинг команд

Функция `parseCommand` анализирует строку и создаёт соответствующий объект команды:

- Определяет тип команды по первому слову
- Для цикла рекурсивно обрабатывает вложенные команды

```
// Парсинг строки в команды
std::unique_ptr<Command> parseCommand(const std::string& line) {
    std::istringstream iss(line);
    std::string token;
    iss >> token;

    if (token == "loop") {
        int iterations;
        iss >> iterations;
        auto loop = std::make_unique<LoopCommand>(iterations);
        std::string subCmd;
        while (std::getline(iss, subCmd, ';')) {
            auto cmd = parseCommand(subCmd);
            if (cmd) loop->addCommand(std::move(cmd));
        }
        return loop;
    }
    else if (token == "print") {
        std::string message;
        std::getline(iss, message);
        return std::make_unique<PrintCommand>(message);
    }
    else if (token == "file") {
        std::string filename, content;
        iss >> filename;
        std::getline(iss, content);
        return std::make_unique<FileAppendCommand>(filename, content);
    }
    else if (token == "math") {
        double a, b;
        char op;
        iss >> a >> op >> b;
        return std::make_unique<MathCommand>(a, b, op);
    }
    return nullptr;
}
```

Параллельное выполнение

- Каждая строка ввода выполняется в отдельном потоке
- Для синхронизации вывода используется мьютекс
- Время начала/окончания работы потоков записывается в консоль

```

void executeCommands(const std::string& line, int lineNum) {
    auto start = std::chrono::steady_clock::now();
    {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << "Thread for line " << lineNum << " started at: "
                    << std::chrono::duration_cast<std::chrono::milliseconds>(start.time_since_epoch()).count()
                    << "ms" << std::endl;
    }

    auto cmd = parseCommand(line);
    if (cmd) cmd->execute();

    auto end = std::chrono::steady_clock::now();
    {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cout << "Thread for line " << lineNum << " finished at: "
                    << std::chrono::duration_cast<std::chrono::milliseconds>(end.time_since_epoch()).count()
                    << "ms" << std::endl;
    }
}

int main() {
    std::vector<std::string> lines;
    std::string line;

    std::cout << "Enter commands (type 'end' to finish):" << std::endl;
    while (std::getline(std::cin, line) && line != "end") {
        lines.push_back(line);
    }

    std::vector<std::thread> threads;
    for (int i = 0; i < lines.size(); ++i) {
        threads.emplace_back(executeCommands, lines[i], i + 1);
    }

    for (auto& t : threads) {
        t.join();
    }

    return 0;
}

```

Заключение

- Реализованы следующие команды:
 1. *print* text - выводит text в консоль. активируется ключевым словом *print*
 2. *math* a operator b - вычисление выражения. активируется ключевым словом *math*. доступные значения operator - +, -, *, /
 3. *file* filename content - записывает content в filename. активируется ключевым словом *file*
 4. *loop* iteration comand1; comand2; ... - выполнение всех перечисленных команд iteration раз. активируется ключевым словом *loop*
- В ходе выполнения ЛР были изучены основы ООП и параллельного программирования в C++. Реализован простой язык программирования с доступным функционалом в виде печати в консоль и в файл, математических операций, а также возможности делать данные операции в циклах с заданным количеством итераций. Реализована параллельная обработка команд, фиксируется время запуска и окончания каждого потока.
- Проверка работы программы

```
Enter commands (type 'end' to finish):
print Hello, World!
math 10 + 15
math 2 - 8
loop 5 print Loop is right!; math 1 + 1
end
Thread for line 1 started at: 1121271463ms
Thread for line 4 started at: 1121271463ms
Thread for line 3 started at: 1121271463ms
Thread for line 2 started at: 1121271463ms
Hello, World!
Thread for line 1 finished at: 1121271466ms
Math: 2 - 8 = -6
Thread for line 3 finished at: 1121271467ms
Loop is right!
Math: 1 + 1 = 2
Loop is right!
Math: 1 + 1 = 2
Loop is right!
Math: 1 + 1 = 2
Loop is right!
Math: 1 + 1 = 2
Loop is right!
Math: 1 + 1 = 2
Thread for line 4 finished at: 1121271470ms
Math: 10 + 15 = 25
Thread for line 2 finished at: 1121271471ms
```